

Oriëntatie op AI

Les 4: algoritmiek - recursie

2025-2026 Semester 1

HBO-ICT

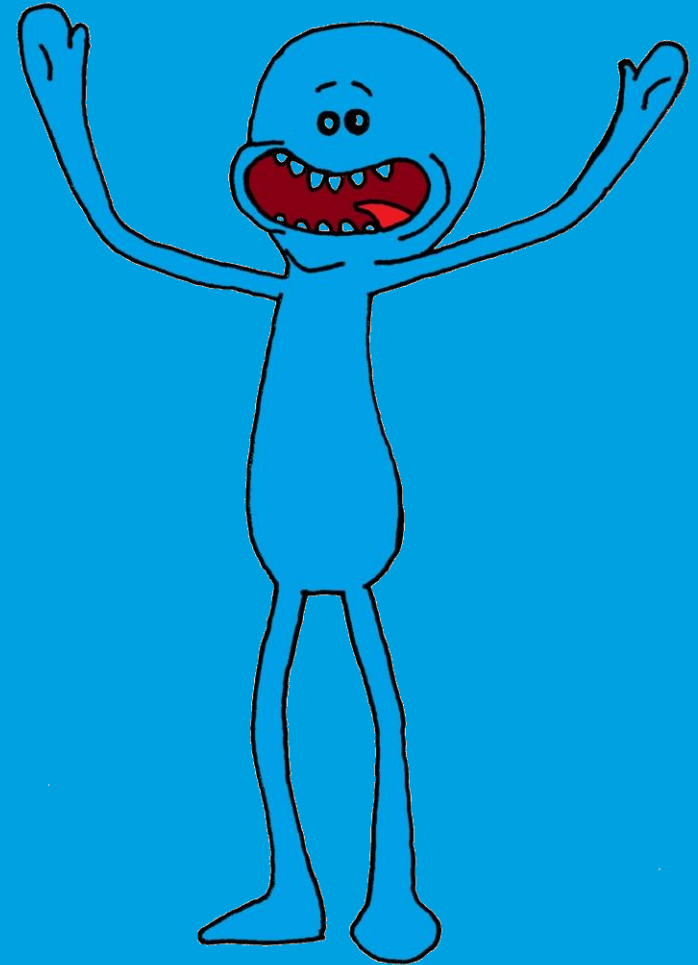


Agenda

1. Recursie
2. Merge sort



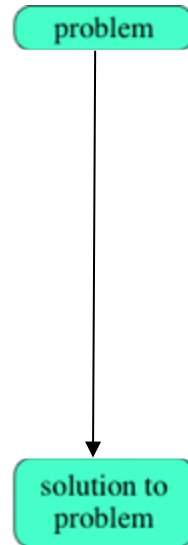
Rekursie



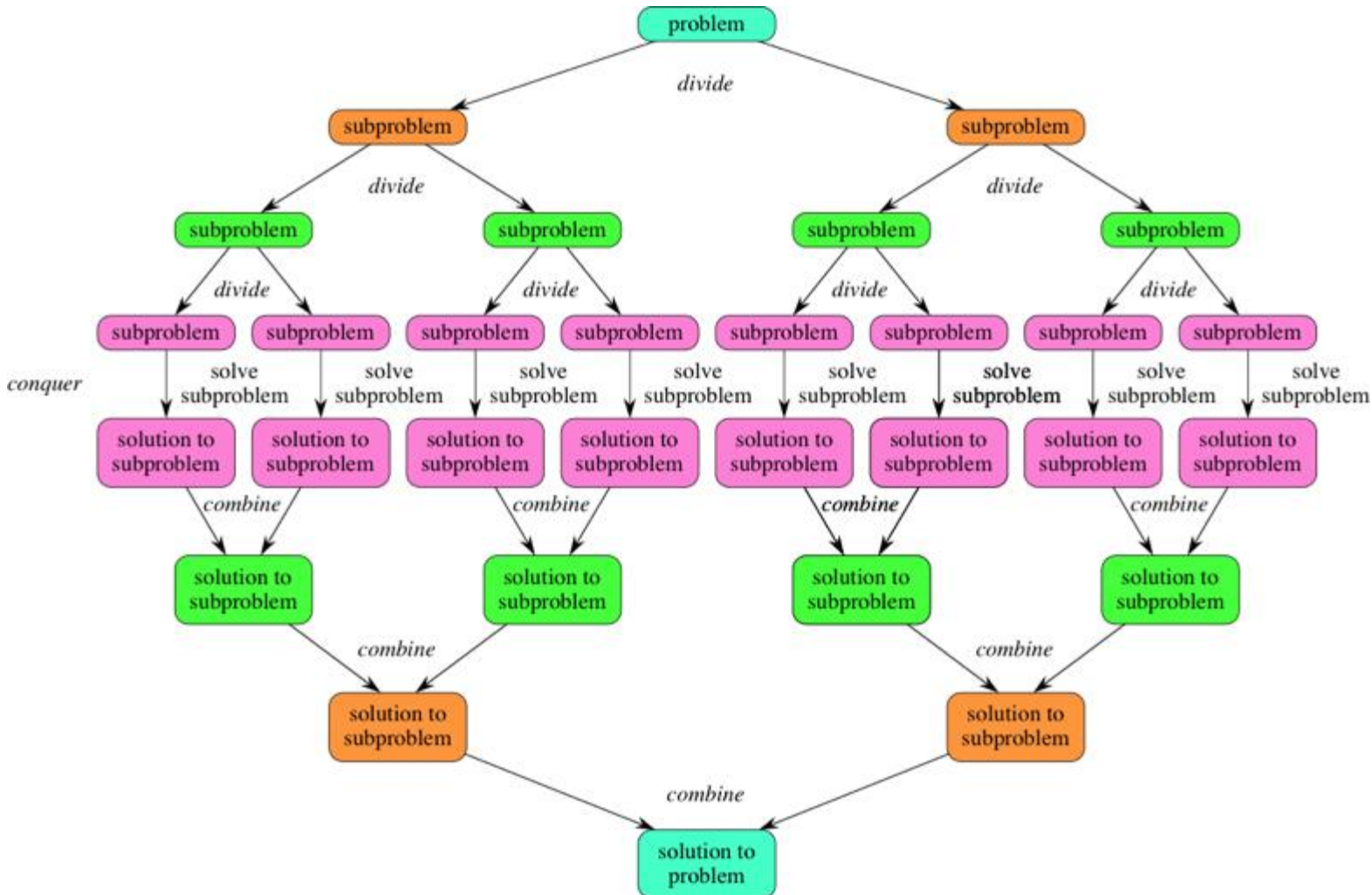
Divide & conquer

1. Verdeel het probleem in een aantal subproblemen die een kleinere instantie zijn van het totaal.
2. Los de subproblemen op als ze klein genoeg zijn. Als de subproblemen niet klein genoeg zijn, verdeel deze weer in subsubproblemen.
3. Combineer de oplossingen van de subproblemen tot de oplossing van het gehele probleem.

Divide & conquer



Divide & conquer



Recursie

- Een algoritme is *recursief* als het een probleem opdeelt in een kleinere variant van hetzelfde probleem, tot het zo klein is dat het direct opgelost kan worden
- Een recursief algoritme *roept zichzelf* aan voor het deelprobleem
- Een recursief algoritme heeft altijd deze twee eigenschappen:
 - *Base case*: de basisvoorwaarde die de oplossing voor het kleinste probleem geeft
 - *Recursion*: aanroep van zichzelf met een kleinere variant van het probleem

Recursie – oefening

- Gegeven onderstaande functie:
 1. **def** functie(x, y) :
 2. **if** x == 1:
 3. **return** y
 4. **return** y + functie(x-1, y)
- Wat is de uitkomst van de aanroep functie(4, 3) ?
- Wat doet deze functie dus eigenlijk?

Recurisie – oefening

```
1. def functie(x, y):  
2.     if x == 1:  
3.         return y  
4.     return y + functie(x-1, y)  
5. print(functie(4, 3))
```

Return: $3 + \text{functie}(4-1, 3)$

$3 + 3 + \text{functie}(3-1, 3)$

$3 + 3 + 3 + \text{functie}(2-1, 3)$

$3 + 3 + 3 + 3$

$= 12$

$x = 4, y = 3$

$x = 3, y = 3$

$x = 2, y = 3$

$x = 1, y = 3$

- Het is een recursieve functie om te vermenigvuldigen: $x \times y$ (als herhaald optellen)



Recursie

- Twee eigenschappen:
 - *Base case*: de basisvoorwaarde die de oplossing voor het kleinste probleem geeft
 - *Recursion*: aanroep van zichzelf met een kleinere variant van het probleem

```
1.  def multiply(x, y):  
2.      if x == 1:  
3.          return y  
4.      return y + multiply(x-1, y)
```

Base case

Recursion

Recurisie – faculteit

- Faculteit: $n!$ is het product van 1 t/m n : $1 \times 2 \times \dots \times n = 1 \times n$
 - $5! = 1 \times 2 \times 3 \times 4 \times 5 = 120$
 - Per definitie geldt $0! = 1$
 - Wordt gebruikt voor permutaties en combinaties
- Oefening: implementeer faculteit op **iteratieve** wijze
 - Oefening AI4.1: Faculteit (iteratief) – (zie Canvas)

Rekursie – faculteit

- Recursiviteit is mooi in de faculteit-functie te zien:
 - $5! = 5 \times 4 \times 3 \times 2 \times 1$
 - $4! = 4 \times 3 \times 2 \times 1$
 - $5! = 5 \times 4!$
 - $4! = 4 \times 3!$
 - ...
 - $1! = 1$
 - $0! = 1$
- De *base case* is $0! = 1$ (of misschien $1! = 1$?)
- De recursie (*recursion*) is $n! = n \times (n - 1)!$

Recursie – oefening

- Schrijf **recursieve** algoritmes in Python voor onderstaande problemen:
 - Gegeven n , bereken $n!$
 - Gegeven n , bereken 2^n ($= 2 \times 2 \times \dots \times 2$, exponent)
 - Gegeven lijst L met getallen, bereken de som van alle elementen van L
 - Gegeven een string, bepaal of de string een palindroom is (spiegelwoord, bijvoorbeeld 'raar' en 'maandnaam')
- Oefening AI4.2: Recursieve algoritmes (zie Canvas)

Merge sort

Merge sort

- Sorteren volgens *divide & conquer*
 - Splits een lijst in twee sublijsten
 - Sorteert de sublijsten
 - Voegt de gesorteerde sublijsten samen

Merge sort

- Sorteren volgens *divide & conquer*
 - Splits een lijst in twee sublijsten
 - Sorteert de sublijsten
 - Splits de sublijst in twee subsublijsten
 - Sorteert de subsublijsten
 - Voeg de gesorteerde subsublijsten samen
 - Voeg de gesorteerde sublijsten samen

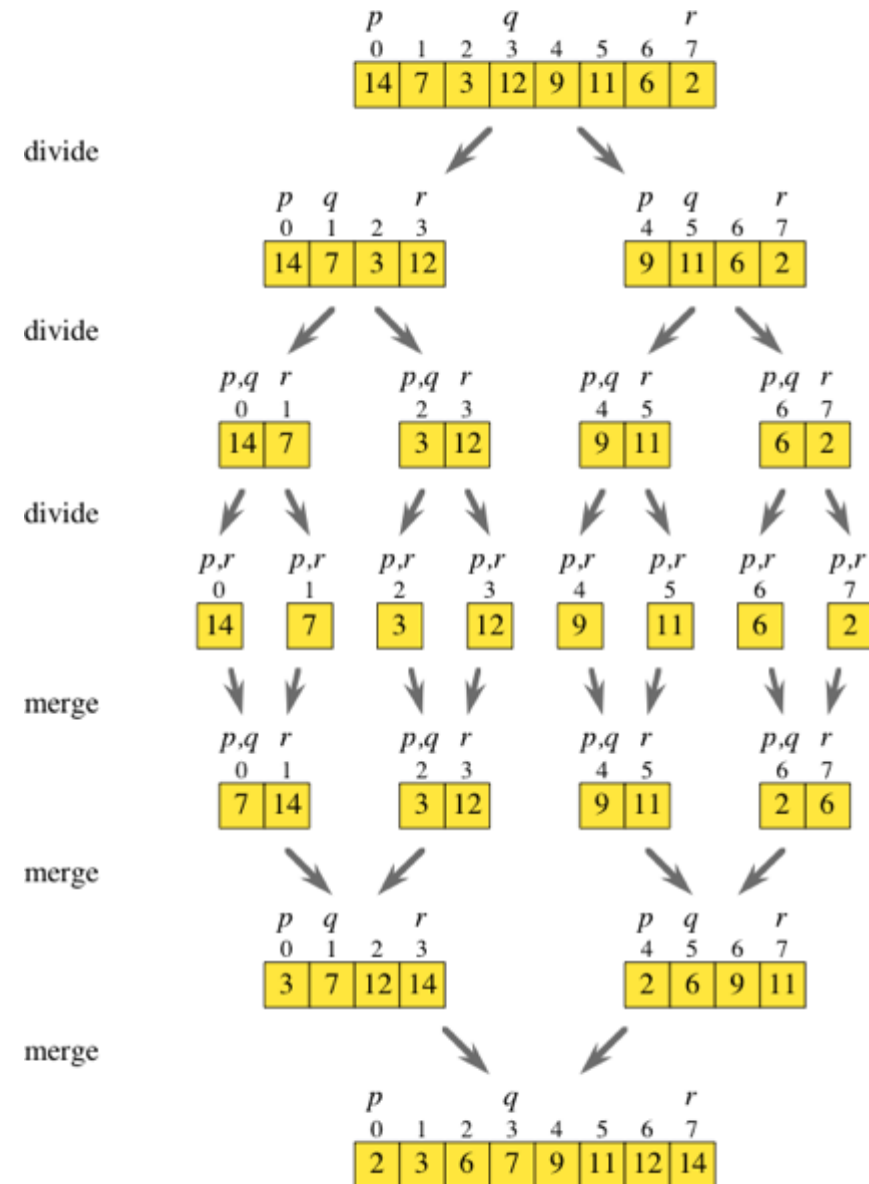
Merge sort

- Sorteren volgens *divide & conquer*
 - Splits een lijst in twee sublijsten
 - Sorteert de sublijsten
 - Splits de sublijst in twee subsublijsten
 - Sorteert de subsublijsten:
 - Splits de subsublijsten in twee subsubsublijsten
 - Sorteert de subsubsublijsten
 - Voer de gesorteerde subsubsublijsten samen
 - Voeg de gesorteerde subsublijsten samen
 - Voeg de gesorteerde sublijsten samen

Merge sort

- Sorteren volgens *divide & conquer*
 - Splits een lijst in twee sublijsten
 - Sorteert de sublijsten
 - Splits de sublijst in twee subsublijsten
 - Sorteert de subsublijsten:
 - Splits de subsublijsten in twee subsubsublijsten
 - Sorteert de subsubsublijsten
 - ...
 - Voert de gesorteerde subsubsublijsten samen
 - Voegt de gesorteerde subsublijsten samen
 - Voegt de gesorteerde sublijsten samen

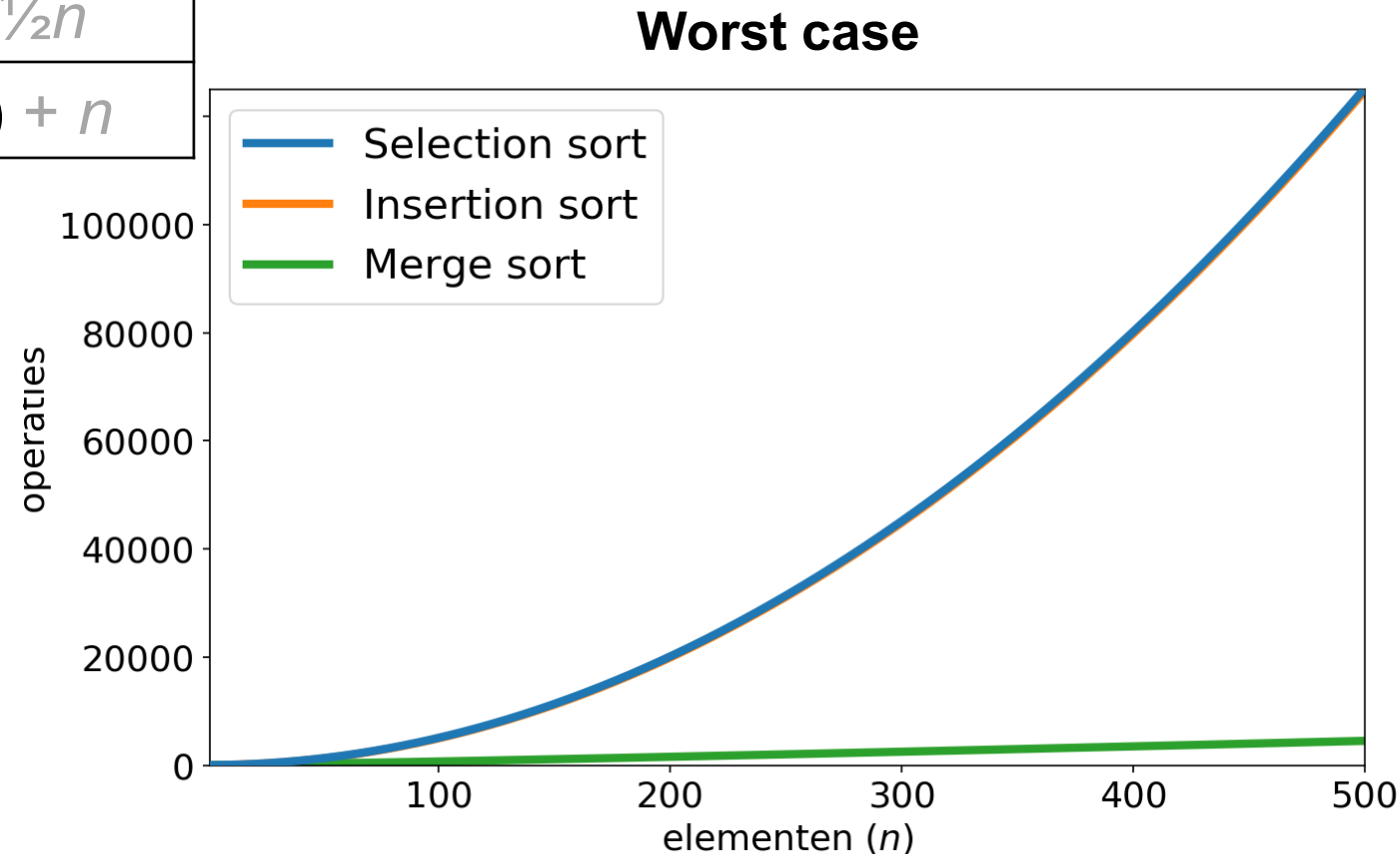
Merge sort



Complexiteiten sorteeralgoritmen

Methode	Best case	Worst case
Selection	$\frac{1}{2}n^2 + \frac{1}{2}n$	$\frac{1}{2}n^2 + \frac{1}{2}n$
Insertion	$n - 1$	$\frac{1}{2}n^2 - \frac{1}{2}n$
Merge	$n \log(n) + n$	$n \log(n) + n$

Voor de gemotiveerde student: analyse op de Khan Academy



Huiswerk

- Maak de recursieve algoritmes af
 - Oefening AI4.2: Recursieve algoritmes (zie Canvas)
- Werk verder aan Opdracht AI1: Algoritmiek